stops processing records every time the transaction is committed. Committing the offset of the processed records locks the stream from further processing.

You can run `ConsumerProducerEffectivelyOnce` as follows:

```
sbt "chapter14/runMain ConsumerProducerEffectivelyOnce 12345"   ❶
```

❶ 12345 is the transactional ID.

If you don’t want to face the drawbacks of a transactional Kafka approach but still need the effectively-once semantics, you can defer the effectively-once semantics in your processing chain. You can defer deduplication to the last component in your infrastructure.

Alpakka Kafka is used again in chapter 15, but the following two connectors—CSV and S3—do not appear in the rest of the book. However, these two connectors are among the most used.

14.4 Alpakka CSV

To read a CSV file, it must contain a delimiter that you can set when parsing the lines, and it deserializes/serializes only UTF-8, ISO-8859-1, and ASCII. You need the following dependencies in your build.sbt file:

```
val AkkaStreamAlpakka = "4.0.0"
val AkkaVersion = "2.6.20"

libraryDependencies ++= Seq(
  "com.lightbend.akka" %% "akka-stream-alpakka-csv" % AkkaStreamAlpakka,
  "com.typesafe.akka" %% "akka-stream" % AkkaVersion
)
```